



BSc (Hons) in **Computer Science**
SE3062 : Intelligent Systems

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 02

Objective & Lecture Mapping: In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{ 'wall_ahead': True/False, 'food_here': True/False }` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*

Example Logic: IF food_here THEN suck; IF wall_ahead THEN turn_left; ELSE move_forward

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent`.
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)`, first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
Example Logic: IF wall_ahead AND left_is_visited THEN turn_right
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

1. Combinatorial Explosion: The number of possible percept sequences and game states is astronomically massive (e.g., 10^{120} possible chess games), making it impossible to pre-compute or store a complete lookup table.
2. Lack of Generalization: It treats every state independently and cannot adapt to novel, unrecorded situations.
3. As the agent's lifetime increases, the required table size grows beyond physical memory limits and computational feasibility, making the agent completely impractical for complex, long-running tasks.

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent`. Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

```
if percept.get('food_here'):
    return 'Suck'
elif percept.get('wall_ahead'):
    return 'TurnLeft'
else:
    return 'MoveForward'
```

Conditions (IF): The expressions evaluated in the conditional statements (`percept.get('food_here')` and `percept.get('wall_ahead')`) represent the agent's current sensory inputs (the condition).

Actions (THEN): The return values ('Suck', 'TurnLeft', 'MoveForward') represent the physical behavior the agent triggers in response to that specific condition.

Stateless Execution: These rules operate strictly on the current percept without checking history or internal memory, matching the direct mapping definition of a Simple Reflex architecture.

3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

The `SimpleReflexAgent` gets stuck in an infinite loop due to two fatal flaws: Partial Observability: The agent's local sensors only reveal immediate surroundings, hiding dead ends or layout traps until it is already inside them. Lack of Percept History: Because it has no memory, it cannot recognize that it has already visited the exact same state moments prior. When trapped, every identical percept triggers the exact same reactive rule, locking the agent into an endless, repetitive cycle: $\text{Hit Wall} \rightarrow \text{Turn Left} \rightarrow \text{Move Forward} \rightarrow \text{Hit Wall} \rightarrow \dots$

4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent` . Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

The `ModelBasedAgent` handles internal state tracking through three key components:

Transition Model: Predicts how the agent's actions change its position (`self.estimated_pos`) and heading internally over time.

Sensor Model: Uses external feedback (such as `hit_wall`) to reconcile internal predictions with physical reality, preventing false position updates when blocked.

Pre-Evaluation State Update: Records coordinates into `self.visited_cells` before running rule checks, allowing the agent to detect loops and choose alternative paths to escape.

Finalize and Lock Lab 02 Documentation



FACULTY OF COMPUTING